Session 6: Ensemble Methods

Javier Serrano
Applied Machine Learning
Master in Data Science and Analytics

Strathmore University
@iLabAfrica Centre

Objectives

- Study a weak classifier like Decision Trees
- Understand that a set of weak predictors can make a robust one.
- Make in depth use of Random Forest as an example of bagging method
- Compare techniques of bagging, boosting and stacking

Sumary

Decision Tree Induction: An Example

Buy a computer?

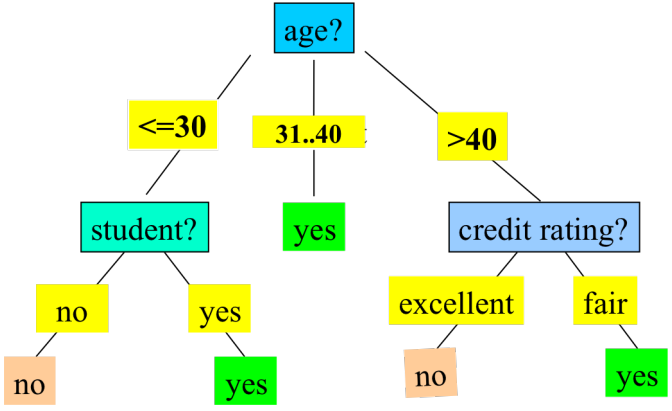
Training data set

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Decision Tree Induction: An Example

Buy a computer?

Resulting tree



Decision Tree Induction

Basic algorithm (a greedy algorithm)

- Tree is constructed in a **top-down recursive divide-and-conquer manner**
- At start, all the training examples are at the root
- Attributes are categorical (if continuous-valued, they are discretized in advance)
- Examples are partitioned recursively based on selected attributes
- Test attributes are selected on the basis of a heuristic or statistical measure (e.g., **information gain, Gini index**)
- Conditions for stopping partitioning
  - All samples for a given node belong to the same class
  - There are no remaining attributes for further partitioning – majority voting is employed for classifying the leaf
  - There are no samples left

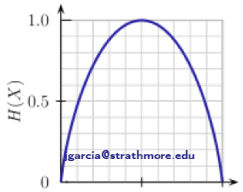
Brief Review of Entropy

Entropy (Information Theory)

- A measure of uncertainty associated to a random variable
- Calculation: For a discrete random variable Y taking m distinct values $\{y_1, \ldots, y_m\}$

$$H(Y) = -\sum_{i=1}^{m} p_i \log(p_i), \text{where} \quad p_i = P(Y = y_)$$

- Interpretation:
  - Higher entropy => higher uncertainty
  - Lower entropy => lower uncertainty



Attribute Selection Measure

Information Gain

- Select the attribute with the highest information gain
- Let p_i be the probability that an arbitrary tuple in D belongs to class C_i, estimated by $|C_{i,D}|/|D|$
- **Expected information** (entropy) needed to classify a tuple in D:

$$\text{Info}(D) = -\sum_{i=1}^{m} p_i \log_2(p_i)$$

- **Information** needed (after using A to split D into v partitions) to classify D:

$$\text{Info}_A(D) = -\sum_{i=1}^{v} \frac{|D_j|}{|D|} \times \text{Info}(D_j)$$

- **Information gained** by branching on attribute A

$$\text{Gain}(A) = \text{Info}(D) - \text{Info}_A(D)$$

Attribute Selection Measure

- Class P: buys_computer = "yes"
- Class N: buys_computer = "no"

$$Info(D) = I(9,5) = -\frac{9}{14}\log_2(\frac{9}{14}) - \frac{5}{14}\log_2(\frac{5}{14}) = 0.940$$

age	p_i	n_i	$I(p_i, n_i)$
<=30	2	3	0.971
31…40	4	0	0
>40	3	2	0.971

$$Info_{age}(D) = \frac{5}{14}I(2,3) + \frac{4}{14}I(4,0) + \frac{5}{14}I(3,2) = 0.694$$

$\frac{5}{14}I(2,3)$ means "age <=30" has 5 out of 14 samples, with 2 yes's and 3 no's. Hence

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

$$Gain(age) = Info(D) - Info_{age}(D) = 0.24$$

Similarly,

$$Gain(income) = 0.029$$
$$Gain(student) = 0.151$$
$$Gain(credit\_rating) = 0.048$$

Attribute Selection Measure

Computing Information-Gain for Continuous-Valued Attributes

- Let attribute A be a continuous-valued attribute
- Must determine **the best split point** for A
  1. Sort the value A in increasing order
  2. Typically, the midpoint between each pair of adjacent values is considered as a possible split point
  3. $(a_i + a_{i+1})/2$ is the midpoint between the values of a_i and a_{i+1}
  4. The point with the minimum expected information requirement for A is selected as the split-point for A
- Split: D1 is the set of tuples in D satisfying A $\leq$ split-point, and D2 is the set of tuples in D satisfying A $>$ split-point

Attribute Selection Measure

Other Attribute Selection Measures

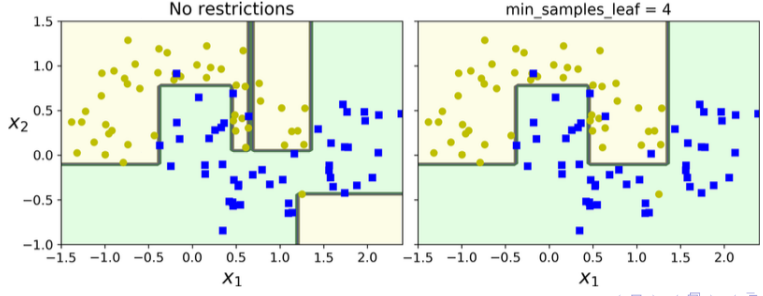
A part of Information Gain there are two more measures that return good results

- **Gain ratio**:
  $\Rightarrow$ tends to prefer unbalanced splits in which one partition is much smaller than the others
- **Gini index**:
  - biased to multivalued attributes (as Information Gain)
  - has difficulty when number of classes is large
  - tends to favor tests that result in equal-sized partitions and purity in both partitions

Decision Trees

Regularization Parameters

- Decision Trees make very few assumptions about the training data (**Non parametric model**)
- The tree structure will adapt itself to the training data, fitting it very closely. (**High Variance - Overfitting**)
- Necessity of **regularization**. The main regularization parameter is *Max_ Depth*



Sumary

Ensemble Methods

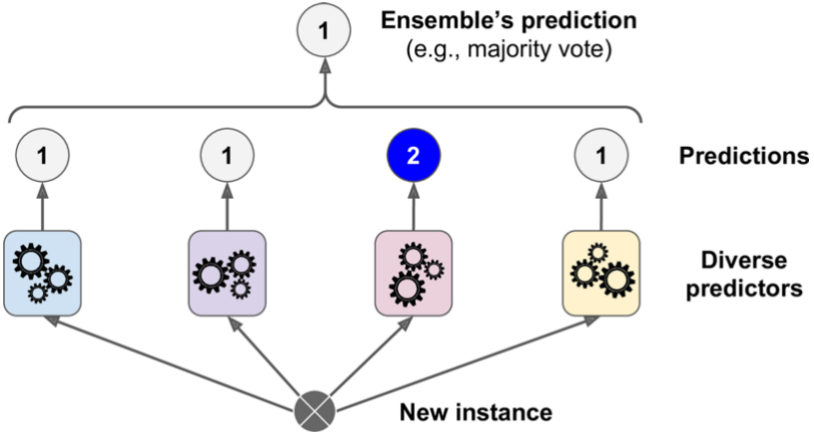
Types

If you aggregate the predictions of a group of predictors (such as classifiers or regressors), you will often get better predictions than with the best individual predictor.

- **Bagging :** Use the same training algorithm for every predictor and train them on different random subsets of the training set.
- **Boosting :** Train predictors sequentially, each trying to correct its predecessor.
- **Stacking :** To aggregate the predictions of all predictors in an ensemble we train a model to perform this aggregation

Ensemble Methods

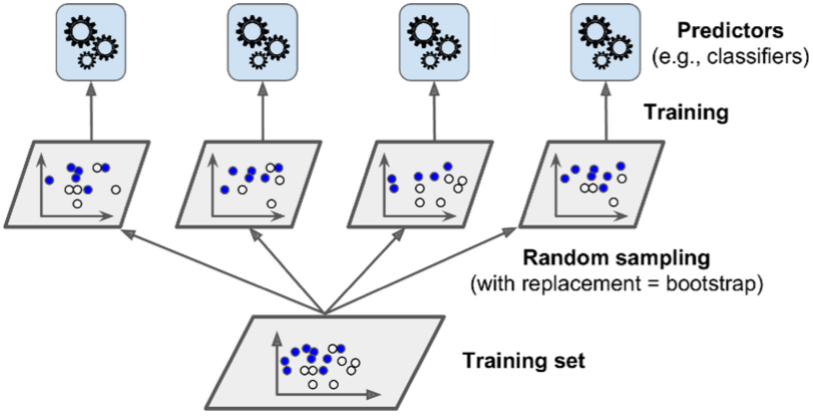
Idea: Hard voting classifier predictions



Sumary

Bagging

Train several weak predictors on different random samples



Bagging

Out-of-Bag Evaluation

- Some instances may be sampled several times for any given predictor, while others may not be sampled at all.
- A Bagging method samples m training instances with replacement, where m is the size of the training set (by default 63%).
- The other 37% is named **Out-of-Bag (oob)**. This oob instances are different for every predictor.
- Since a predictor never sees the oob instances during training, it can be evaluated on these instances, without the need for a separate validation set.
- Evaluate the ensemble itself by averaging out the oob evaluations of each predictor.
- To get a more rigorous validation we can use k-fold Cross Validation.

Bagging

Random Subspaces: Sampling Features

- We can sampling the features as well. Each predictor will be trained on a random subset of the input features.
- This technique is particularly useful when you are dealing with high-dimensional inputs (such as images)
- Sampling features results in even more predictor diversity, **better generalization**
- We can mix oob and random subspaces in a *Random Patch*

Bagging

Random Forest (RF)

- RF is an ensemble of Decision Trees, trained via the bagging method.
- RF has all the hyperparameters of a Decision Tre eClassifier (to control how trees are grown), plus all the hyperparameters of a Bagging Classifier to control the ensemble itself. **Good choice as an initial classifier**
- RF introduces extra randomness when growing trees; instead of searching for the very best feature when splitting a node, it searches for the best feature among a random subset of features. **Better generalization and performance**

Random Forest (RF)

Feature Importance

- RF make it easy to measure the relative importance of each feature
- RF measures a feature's importance by looking at how much the tree nodes that use that feature reduce impurity (or increase gain information) on average (across all trees in the forest).
- RF are very handy to get a quick understanding of what features actually matter.

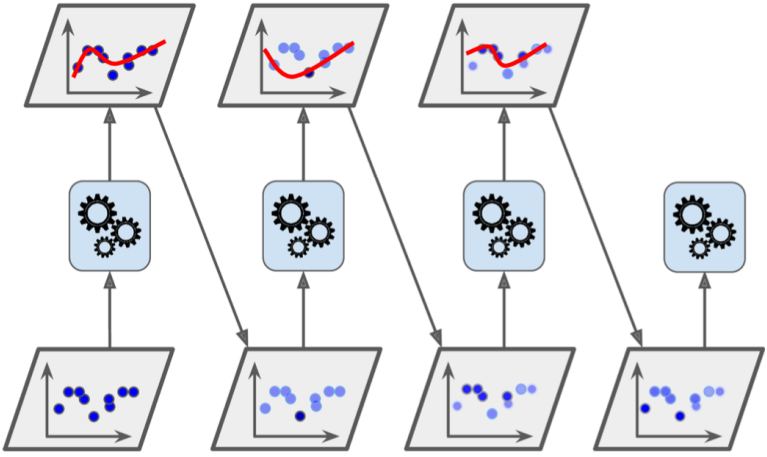
Sumary

Boosting

- Train (weak) predictors sequentially, each trying to correct its predecessor.
- Two strategies:
  1. **AdaBoost**: increases the relative weight of misclassified training instances
  2. **Gradient Boosting**: Fits the new predictor to the residual errors made by the previous predictor.

AdaBoost

Sequential training with instance weight updates



AdaBoost

Algorithm I

1. Each instance weight $w^{(i)}$ is initially set to $1/m$. A first predictor is trained
2. Weighted error rate r_i is computed on the training set

$$r_j = \frac{\sum_{i=1}^{m} w^{(i)}}{\sum_{i=1}^{m} w^{(i)}} \quad \text{where } \hat{y}_j^{(i)} \text{ is the } j^{\text{th}} \text{ predictor's prediction for the } i^{\text{th}} \text{ instance.}$$

3. We compute the predictor weight and then update the instance weight

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

AdaBoost

Algorithm II

$$\text{for } i = 1, 2, \cdots, m$$

$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \text{if } \widehat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} \exp\left(\alpha_j\right) & \text{if } \widehat{y}_j^{(i)} \neq y^{(i)} \end{cases}$$

4. A new predictor is trained using the news weights, and the whole process is repeated.
5. The algorithm stops when the desired number of predictors is reached, or when a perfect predictor is found.

AdaBoost

Making Predictions

- AdaBoost simply computes the predictions of all the predictors and weighs them using the predictor weights α_j.
- The predicted class is the one that receives the majority of weighted votes

$$\hat{y}(\mathbf{x}) = \underset{k}{\operatorname{argmax}} \sum_{\substack{j=1 \\ \hat{y}_j(\mathbf{x}) = k}}^{N} \alpha_j \quad \text{where } N \text{ is the number of predictors.}$$

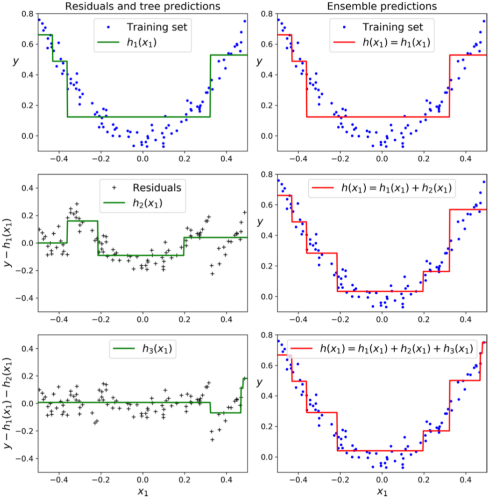
Boosting

Gradient Boosting

- Sequentially adding predictors to an ensemble, each one correcting its predecessor(as AdaBoost)
- Tries to fit the new predictor to the **residual errors** made by the previous predictor.
- Next example shows how *gradient tree boosting* works

Gradient Boosting

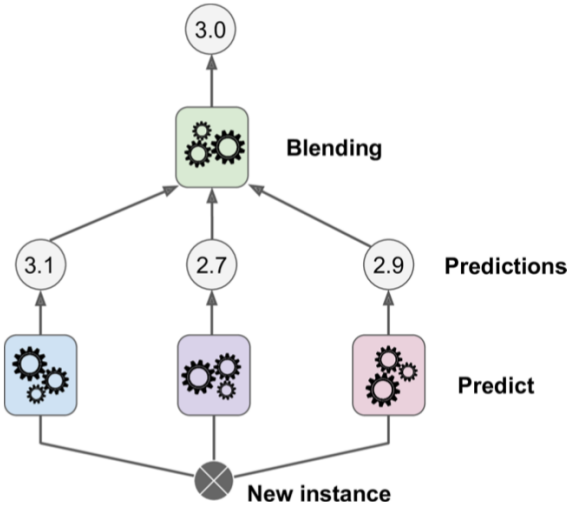
Example



Sumary

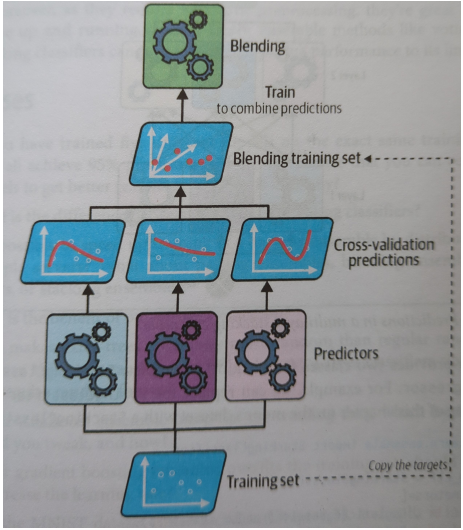
Stacking

Stacked Generalization



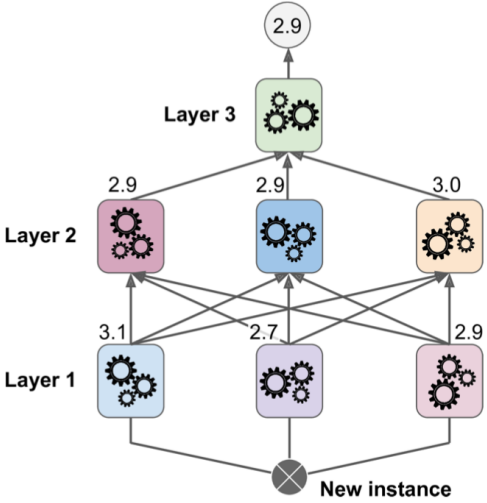
Stacking

Training the blender in a stacking ensemble



Stacking

Predictions in a multilayer stacking ensemble



Sumary

- Decision trees as a simpe and weak predictor
- Ensemble methods as a way of getting robust/**strong predictors**
- Random Forest as an example of Bagging, we **train trees with subsets of training data** and the predictors vote for the best classification. Parallel computing.
- Boosting, **sequencial training** trying to improve the previous predictor. **AdaBoost and Gradient Boosting**.
- **Stacking**. A combination of many good ideas, especially the **blender**.